STEP-BY-STEP GUIDE

Back up and migrate Azure AI Search to a new resource group

Rebuild the service, copy every index definition and move the documents, with commands you can run as you go.

8 steps Azure CLI + REST API + bash Tested end to end on a sample service

BEFORE YOU START

A migration rebuilds three layers

Azure AI Search has no self-service backup or restore, so a copy is rebuilt piece by piece. The source service stays untouched until you retire it in Step 8.

1

Service

Tier, replicas, partitions, region, semantic ranker, managed identity, network rules

Azure CLI · Steps 2–3

2

Definitions

Indexes, synonym maps, data sources, skillsets and indexers, as JSON

REST export and import · Steps 4–5

3

Documents

The indexed content itself

Re-run indexers or dump and reload · Step 6

BEFORE YOU START

Pick a data path for each index

	Option A · Re-run indexers	Option B · Dump and reload
Use when	An indexer fills the index	Your app pushes documents
Copies	Everything, rebuilt from the source data	Only fields marked retrievable
Cost	Skillsets and embeddings run again	Bandwidth and your time
Time	Hours to days for large sets	Minutes to hours
Watch out	A full re-index; change tracking does not carry over	Non-retrievable fields, often vectors, arrive empty

Most services mix both: use A for indexer-fed indexes and B for push indexes. No source data and non-retrievable fields? Those fields can't be recovered.

BEFORE YOU START

What you need before Step 1

- Azure CLI 2.50+ signed in, plus **curl** and **jq** in a bash terminal
- Contributor on the source and destination resource groups
- Admin keys, or Search Service Contributor, on both services
- Connection strings and AI service keys for every data source and skillset; exports blank them
- .NET 8 SDK if you use Microsoft's backup tool for Option B
- A write freeze window if apps push documents during the copy

⚠ On Windows?

Git Bash works, but its jq adds a hidden \r to every line, which breaks object names in URLs.

The setup on the next slide includes a one-line fix.

SETUP · ONCE PER MIGRATION

Save settings once, load per terminal

```
# 1. Save your values (edit the names first)
cat > ~/search-migration.env <<'EOF'
SRC_RG=old-rg       SRC_SVC=old-search
DST_RG=new-rg       DST_SVC=new-search
LOCATION=centralindia API=2024-07-01
SRC=https://old-search.search.windows.net
DST=https://new-search.search.windows.net
EOF
```

```
# 2. Run at the start of EVERY terminal session
set -a; source ~/search-migration.env; set +a
export SRC_KEY=$(az search admin-key show -g $SRC_RG --service-name $SRC_SVC --query primaryKey -o tsv)
jq() { command jq "$@" | tr -d '\r'; }; export -f jq   # Git Bash on Windows only
```

set -a exports every variable, so the scripts in Steps 4–5 can see them. After Step 2, also export DST_KEY the same way.

STEP 1 OF 8

Inventory the source service

```
mkdir -p ~/backup && cd ~/backup
az search service show -g $SRC_RG -n $SRC_SVC > service.json

# list every object
names() { curl -s "$1/$2?api-version=$API&\$select=name" -H "api-key: $3" | jq -r '.value[].name'; }
for t in indexes synonymmaps datasources skillsets indexers; do echo "== $t"; names $SRC $t $SRC_KEY; done

# baseline document counts, used again in Step 7
for i in $(names $SRC indexes $SRC_KEY); do
  echo "$i: $(curl -s "$SRC/indexes/$i/docs/\$count?api-version=$API" -H "api-key: $SRC_KEY")"
done | tee doc-counts.txt
```

service.json holds the tier, replicas, partitions and semantic setting to copy in Step 2.

Also note every app that calls this service; you repoint them in Step 8.

STEP 2 OF 8

Create the destination service

```
az search service create -g $DST_RG -n $DST_SVC -l $LOCATION \
  --sku standard --replica-count 1 --partition-count 1 \
  --semantic-search free --identity-type SystemAssigned \
  --auth-options aadOrApiKey -o table

export DST_KEY=$(az search admin-key show -g $DST_RG --service-name $DST_SVC --query primaryKey -o tsv)
```

Match the source

Copy the tier, replicas, partitions and semantic setting from service.json. The tier must hold the source's storage and object counts.

New name, new endpoint

Service names are global, so clients will need the new URL and key in Step 8.

Expect 3–10 minutes

The resource group must exist first; create it with az group create if needed.

STEP 3 OF 8

Grant identity and network access

```
DST_MI=$(az search service show -g $DST_RG -n $DST_SVC --query identity.principalId -o tsv)
az role assignment create --assignee-object-id $DST_MI --assignee-principal-type ServicePrincipal \
  --role "Storage Blob Data Reader" --scope <storage-account-resource-id>
```

Resource	Role for the new identity
Blob / ADLS storage	Storage Blob Data Reader
Knowledge store, cache	Storage Blob Data Contributor
Azure OpenAI	Cognitive Services OpenAI User
Azure AI services	Cognitive Services User
Azure SQL	Contained user with db_datareader

Also recreate

- IP firewall rules
- Private endpoints and DNS
- Shared private links (approve again)
- App roles for Entra ID clients
- Customer-managed keys

STEP 4 OF 8 · EXPORT-DEFINITIONS.SH

Export every object definition

```
#!/usr/bin/env bash
set -uo pipefail
for t in synonymmaps indexes datasources skillsets indexers; do
  mkdir -p "$t"
  code=$(curl -s -o /tmp/list.json -w '%{http_code}' -H "api-key: $SRC_KEY" \
    "$SRC/$t?api-version=$API&\$select=name")
  [[ $code == 200 ]] || { echo "SKIP $t ($code)"; continue; }
  for n in $(jq -r '.value[].name' /tmp/list.json | tr -d '\r'); do
    code=$(curl -s -o /tmp/obj.json -w '%{http_code}' -H "api-key: $SRC_KEY" \
      "$SRC/$t/$n?api-version=$API")
    if [[ $code == 200 ]]; then
      jq 'del(."@odata.context", ."@odata.etag")' /tmp/obj.json > "$t/$n.json" && echo "exported $t/$n"
    else echo "FAIL $t/$n ($code)"; cat /tmp/obj.json; echo; fi
  done
done
```

Run: chmod +x export-definitions.sh && ./export-definitions.sh — expect one **exported** line per object.

STEP 5 OF 8 · PART 1

Restore secrets, respect the order

Synonym maps → Indexes → Data sources → Skillsets → Indexers

Exported file	Field returned as null	Set it to
datasources/*.json	credentials.connectionString	Connection string, or ResourceId=...; for identity
skillsets/*.json	cognitiveServices.key, apiKey	AI services or Azure OpenAI key
indexes/*.json	vectorizer apiKey	Azure OpenAI key

```
jq --arg cs "$CONN" '.credentials.connectionString = $cs' datasources/my-ds.json > tmp \
  && mv tmp datasources/my-ds.json
grep -rl '"connectionString": null\|"key": null\|"apiKey": null' . || echo "no blank secrets"
```

STEP 5 OF 8 · PART 2 · IMPORT-DEFINITIONS.SH

Import definitions into the new service

```
#!/usr/bin/env bash
set -uo pipefail
for t in synonymmaps indexes datasources skillsets indexers; do
  for f in "$t"/*.json; do
    [ -e "$f" ] || continue
    n=$(basename "$f" .json); body=$(cat "$f")
    [ "$t" = indexers ] && body=$(echo "$body" | jq '.disabled = true')
    code=$(curl -s -o /tmp/resp.json -w '%{http_code}' -X PUT "$DST/$t/$n?api-version=$API" \
      -H "api-key: $DST_KEY" -H 'Content-Type: application/json' -d "$body")
    if [[ $code == 20* ]]; then echo "OK   $t/$n"
    else echo "FAIL $t/$n ($code)"; cat /tmp/resp.json; echo; fi
  done
done
```

Expect one OK line per object. Indexers are created disabled, so nothing runs until Step 6.

A FAIL prints the service's error. Fix that file and rerun; a PUT simply overwrites.

Option A: let indexers rebuild the data

```
for f in indexers/*.json; do n=$(basename "$f" .json)
  jq '.disabled = false' "$f" | curl -s -X PUT "$DST/indexers/$n?api-version=$API" \
    -H "api-key: $DST_KEY" -H 'Content-Type: application/json' -d @- > /dev/null
  curl -s -X POST "$DST/indexers/$n/run?api-version=$API" -H "api-key: $DST_KEY" -H 'Content-Length: 0'
  echo "started $n"
done

# check: the document count, then the whole run history
curl -s "$DST/indexes/<index>/docs/\$count?api-version=$API" -H "api-key: $DST_KEY"
curl -s "$DST/indexers/<indexer>/status?api-version=$API" -H "api-key: $DST_KEY" \
  | jq '[.executionHistory[] | {status, processed: .itemsProcessed}]'
```

> **status: success with processed: 0 is fine** when an earlier run already indexed everything. Trust the count and the history, not only the last run.

Option B: dump and reload documents

```
# small indexes: dump from the source...
curl -s -X POST "$SRC/indexes/<index>/docs/search?api-version=$API" -H "api-key: $SRC_KEY" \
  -H 'Content-Type: application/json' -d '{"search":"*","top":1000}' \
  | jq '{value: [.value[] | del."@search.score") | . + {"@search.action":"upload"}]}' > docs.json
# ...and upload to the destination (expect a list of true)
curl -s -X POST "$DST/indexes/<index>/docs/index?api-version=$API" -H "api-key: $DST_KEY" \
  -H 'Content-Type: application/json' -d @docs.json | jq -c '[.value[].status]'
```

Larger indexes

Use Microsoft's backup tool: clone Azure-Samples/azure-search-dotnet-utilities, fill appsettings.json in index-backup-restore, run dotnet run once per index.

What it can't copy

Fields with retrievable: false come back empty. The curl shortcut reads only 1,000 documents, and search can't page past 100,000.

STEP 7 OF 8

Validate before you switch

```
# compare counts with the Step 1 baseline; expect "counts match"
for i in $(curl -s "$DST/indexes?api-version=$API&\$select=name" -H "api-key: $DST_KEY" | jq -r
'.value[].name'); do
  echo "$i: $(curl -s "$DST/indexes/$i/docs/\$count?api-version=$API" -H "api-key: $DST_KEY")"
done | diff doc-counts.txt - && echo "counts match"
```

- ✓ Object lists match Step 1
- ✓ Same top results for sample queries
- ✓ Indexers succeed and keep their schedules
- ✓ Document counts match
- ✓ Semantic, vector and synonym queries work
- ✓ Clients can query with the new key or role

Finish Step 6 for every index first. A push index shows 0 until Option B has run for it; that is a skipped step, not a failure.

STEP 8 OF 8

Cut over, then retire the source

1

Freeze writes

Pause pushes to the source, or write to both, so no update is lost.

2

Repoint clients

New endpoint and key or role in app settings, Key Vault, Azure OpenAI and AI Foundry connections, Logic Apps.

3

Stop the source

Disable its indexers so enrichment isn't paid for twice.

4

Watch, back up, delete

Monitor latency and throttling for a few days, take a final backup, then delete.

```
# irreversible: a deleted service or index can't be recovered
az search service delete -g $SRC_RG -n $SRC_SVC --yes
```

LESSONS FROM A TEST RUN

Four errors from testing, and fixes

What you see	Why	Fix
SRC: unbound variable	Variables are set but not exported; ./script runs in a child shell	set -a; source ~/search-migration.env; set +a
HTTP 000, file named docs-blob$'\r'.json	Git Bash jq ends lines with \r\n	jq() { command jq "$@" | tr -d '\r'; }; export -f jq
Empty output from a pasted one-liner	A backslash before a space drops the -H header	Remove the \ when joining lines into one
Indexer success, processed: 0	An earlier run already did the work	Check docs/$count and executionHistory

All four fixes are already built into the setup and scripts in this guide.

REFERENCE

Troubleshooting and rollback

Symptom	Likely cause	Fix
403 on a REST call	Wrong key, or local auth disabled	Reload the key, or send an Entra bearer token
Index PUT: quota error	Destination tier too small	Raise tier or partitions, rerun import
Index PUT: semantic config	Semantic ranker off	az search service update --semantic-search free
Data source credentials error	Secret still null, or role missing	Redo Step 5 part 1 or Step 3; roles take ~10 min
Skill fails in indexer	AI key or OpenAI role missing	Fill apiKey or grant OpenAI User
Restored vectors empty	Field not retrievable	Use Option A for that index

> **Rollback, any time before the final delete:** point clients back to the source and delete only the new service. Never delete a shared resource group.

RECAP

The whole migration on one page

1
Inventory
Settings, objects, counts

2
Create service
Same tier and features

3
Access
Identity roles, network

4
Export
Definitions to JSON

5
Import
Secrets back, in order

6
Move data
Indexers or dump and reload

7
Validate
Counts and queries match

8
Cut over
Repoint, then retire

Keep the backup folder: the exported definitions plus dumped documents are your point-in-time backup for next time.